

Account Demolisher: Technical Architecture Document

Submission: Account Demolisher (SCF Build, RFP Track)

Written by: Salih Toruner - salihTORUNR@gmail.com

SCF Build Integration List:

<https://stellar.gitbook.io/scf-handbook/scf-awards/scf-build/integration-track/integration-list>

1. Purpose and scope

Account Demolisher is a production-grade, non-custodial web tool that takes a Stellar account from any starting state to cleanly closed in one guided session, recovering the XLM that subentries and the base reserve would otherwise leave locked. It covers the full classic surface and, unlike the existing [stellar.expert/demolisher](#), the Soroban surface: smart-contract token balances, SEP-41 allowances, and DeFi positions across Blend, Aquarius, Soroswap, and FxDAO.

This is an RFP Track submission, so the document is Stellar-specific throughout and gives concrete integration detail: the exact SDKs, mainnet and testnet contract addresses, Stellar operations, Horizon and Soroban RPC endpoints, transaction sequences, and signing flows the tool uses. Section 3 maps each integration onto the SCF Build Integration List building blocks the project has chosen; the remaining sections specify how each is integrated and how the pieces fit together. The architecture described here is the design to be built into the production tool over the grant period; a working, open-source prototype already exercises these flows end-to-end on testnet, with its source at <https://github.com/bytemaster333/account-demolisher> and a live demo at <https://demolisher.saliht.xyz/>, and that prototype is the basis for the feasibility claims; the production tool is the funded deliverable.

The tool integrates four building blocks from the Integration List, namely **Stellar Wallets Kit**, **Blend v2**, **Aquarius**, and **Soroswap**. It also consumes the RFP-specified DeFi Position API backend, **Orion** or **OctoPos**, both SCF #41 awards, and it unwinds **FxDAO** vaults through direct Soroban contract calls. Per the Integration List guidance, the bulk of the requested budget goes to this integration work.

2. System overview

The tool is a statically pre-rendered Next.js application written in TypeScript, paired with an intentionally thin backend. It reads account and protocol state through Horizon and Soroban RPC and writes only transactions the user signs on their own device. There is no database, no user-session store, and no analytics.

Two Stellar protocol facts determine the whole execution model:

1. `ACCOUNT_MERGE` is the last operation an account can perform. It requires the High threshold and fails with `ACCOUNT_MERGE_HAS_SUB_ENTRIES` while any subentry remains, `ACCOUNT_MERGE_IS_SPONSOR` while the account sponsors reserves for another (`numSponsoring > 0`), and `ACCOUNT_MERGE_IMMUTABLE_SET` permanently if the `AUTH_IMMUTABLE` flag is set, which the tool must detect pre-flight.
2. A Soroban transaction may contain exactly one `InvokeHostFunctionOp`, and Soroban and classic operations cannot be mixed in one transaction. Each contract call is therefore its own transaction with its own sequence-number bump.

An account that holds Soroban positions is closed with an N+1 transaction plan: N single-call Soroban transactions to exit positions, revoke allowances, and drain contract tokens, followed by one classic transaction (up to 100 operations) that clears the remaining subentries and performs the merge. Because the steps are sequential and state-dependent, the tool re-reads on-chain state before building each step, which makes the live ledger the source of truth and the whole sequence idempotent.

The trust model follows from the tool's purpose. Account closure is the most destructive action a user can take on Stellar, so the design minimizes the trust it requires. The user's secret key never leaves the client. The only secret the server holds is a per-deployment mediator account seed, whose sole privileged action is co-signing a strictly shape-validated two-operation envelope. Every Soroban transaction is checked against a hardcoded, network-specific contract allow-list before signing, and the full plan is previewed before anything is signed. The tool refuses to attempt anything it cannot complete safely.

3. Stellar integration map

The first four rows are SCF Build Integration List items. The Orion / OctoPos row is the DeFi Position API backend the RFP names explicitly. The FxDAO row is an in-scope protocol integrated by direct contract call.

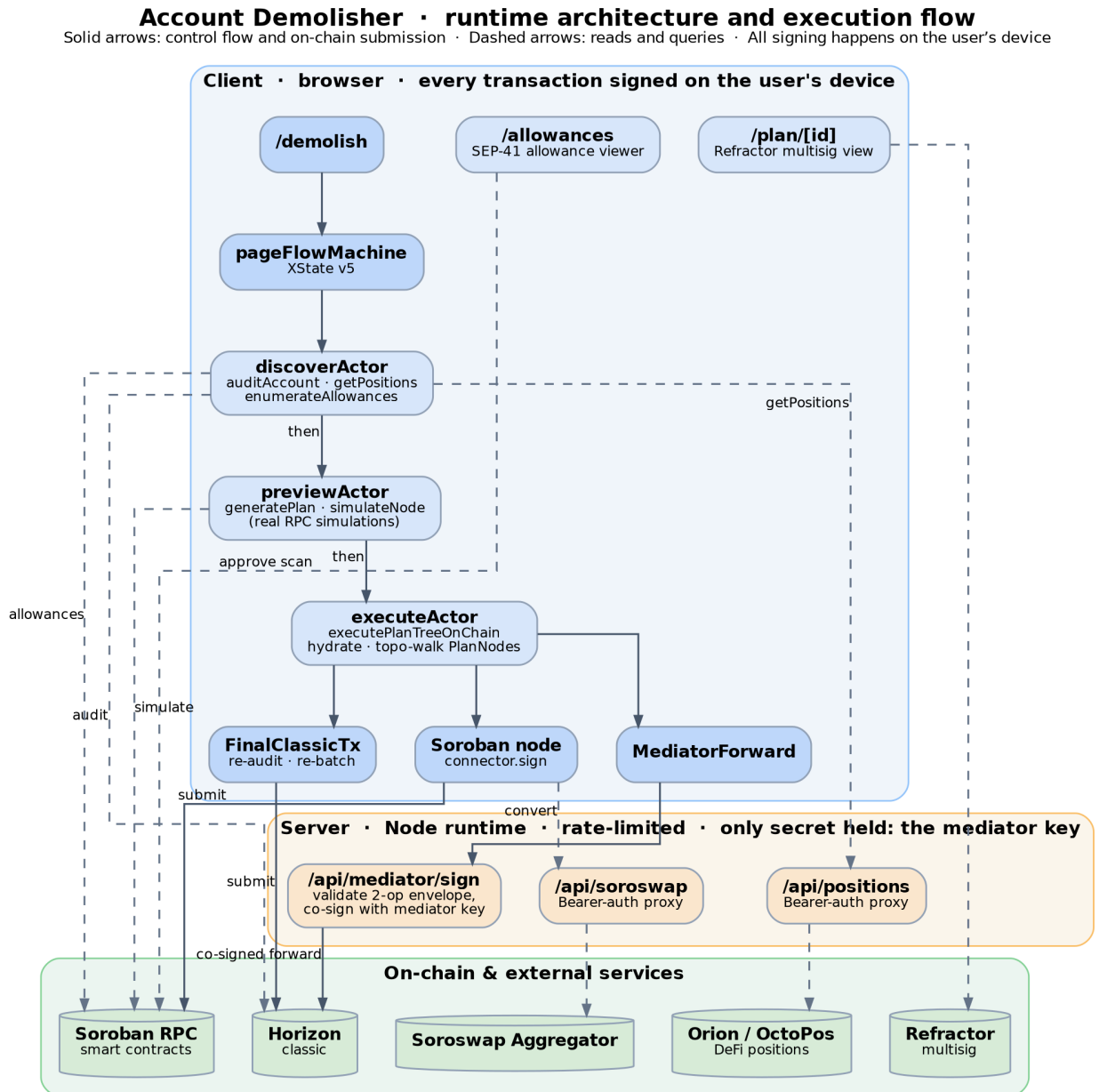
Building block	Integration List category	How it is integrated	SDK / interface	Stellar surface touched	Milestone
Stellar Wallets Kit	Wallet Connection Layers	Default wallet connection and per-step signing; multisig coordination	@creit.tech/stellar-wallets-kit 2.3.0 (SEP-43 signTransaction, signAuthEntry)	Classic and Soroban transaction signing across Freighter, Ledger, xBull, Albedo, Rabet, LOBSTR, Hana, WalletConnect	MVP (T1), extended T2
Blend v2	DeFi, Individual Protocols	Read lending positions; repay liabilities, withdraw collateral and supply, claim emissions, queue backstop withdrawal	@blend-capital/blend-sdk 3.2.2 (PoolV2.load, PoolContractV2.submit, RequestType)	Soroban contract calls, one InvokeHostFunctionOp per step	Testnet (T2)
Aquarius	DeFi, Individual Protocols	Withdraw AMM liquidity, claim AQUA, route swaps	Direct contract invocation via @stellar/stellar-sdk plus the public	Soroban contract calls to the Aquarius router	Testnet (T2)

Building block	Integration List category	How it is integrated	SDK / interface	Stellar surface touched	Milestone
			<code>amm-api.aqua.network</code> for pool discovery		
Soroswap	DeFi, Individual Protocols	Remove LP; convert any Soroban token to XLM via the multi-protocol Aggregator	<code>@soroswap/sdk 0.4.0</code> (<code>quote</code> , <code>build</code> , <code>removeLiquidity</code>)	Soroban contract calls; Aggregator routes across Soroswap, Phoenix, Aquarius	Testnet (T2)
Orion / OctoPos (RFP-named backend)	DeFi position data (SCF #41)	Discover DeFi positions across protocols from one address, behind a provider abstraction with a direct on-chain fallback	REST/JSON, proxied server-side; <code>IDeFiPositionProvider</code> adapters	Read-only position discovery	Testnet (T2)
FxDAO (in-scope protocol)	DeFi (not on the quarterly list)	Repay vault debt, redeem collateral, dispose of the leftover classic stablecoin	Direct contract invocation via <code>@stellar/tellar-sdk</code>	Soroban vault calls plus classic disposal of USDx / EURx / GBPx	Testnet (T2)

Position discovery is deliberately not rebuilt: the tool consumes the SCF-funded Orion or OctoPos API and falls back to direct on-chain reads only where the API does not yet cover a protocol, which keeps the tool functional regardless of external API readiness and makes it a clean reference consumer for whichever matures first.

4. Component and module architecture

The application is organized so that every Stellar-touching concern is an isolated, independently testable module. The user-facing flow is an XState v5 state machine; the privileged operations live behind three server routes; everything else is a typed library module.



The library is laid out by responsibility:

- **lib/stellar**: classic account audit, the allow-list check, claimable-balance handling, the classic transaction builder, the Horizon client, and classic pathfinding.
- **lib/soroban**: the RPC client, ScVal constructors, the SEP-41 helper set, transaction simulation, allowance enumeration, the SAC helper, and the spender registry.
- **lib/adapters**: one per-protocol exit module for Blend, Aquarius, Soroswap, FxDAO, plus the position-provider adapters.
- **lib/plan**: the plan generator, the classic operation batcher, plan hydration, the per-node simulator, the plan tree, and the classic orchestrator.
- **lib/orchestrator**: the XState machine, the page-flow machine, the typed recovery dispatch, and the state definitions.
- **lib/multisig**: the signer-requirement inspector, the Refractor client, and the partial-XDR signature merger.
- **lib/mediator**: the envelope client, the strict validator, and the forward step.
- **lib/safety**: the CEX registry, fee-surge protection, memo enforcement, scam-token heuristics, and slippage policy.
- **lib/wallet**: the wallet connector interface, the Stellar Wallets Kit adapter, and the secret-key fallback.
- **server/**: the server-only mediator-secret loader and the Zod-validated environment schema.

5. Technology stack and dependency lockset

The dependency set is pinned and verified against the npm registry, and the lockfile is committed and installed with **--frozen-lockfile** in CI. The stack matches the existing tool family (React and Next.js) and uses one Stellar SDK for both Horizon and Soroban.

Layer	Pin	Notes
Runtime	Node.js 22 LTS	Active LTS
Package manager	pnpm 10.30.0	Lockfile pinned, CI frozen
Language	TypeScript 6.0.3	strict , noUncheckedIndexedAccess , exactOptionalPropertyTypes
Framework	Next.js 16.2.6	App Router, static export for the trust-critical pages

Layer	Pin	Notes
UI	React 19.2.6, Tailwind CSS 4.3.0, shadcn/ui 4.7.0	
State	Zustand 5.0.13 (global), XState 5.31.1 (orchestrator)	
Validation	Zod 4.4.3	Environment and adapter-response schemas
Stellar SDK	@stellar/stellar-sdk 14.4.3	Pinned to match the Blend SDK's exact dependency so there is no duplicate SDK; v14 exposes the <code>rpc</code> module with <code>rpc.Server.prepareTransaction</code> and <code>pollTransaction</code>
Wallets	@creit.tech/stellar-wallets-kit 2.3.0	Freighter, xBull, Albedo, Rabet, LOBSTR, Hana, WalletConnect v2
Blend	@blend-capital/blend-sdk 3.2.2	<code>PoolV2.load</code> , <code>PoolContractV2.submit</code> , <code>RequestType</code>
Soroswap	@soroswap/sdk 0.4.0	Pre-1.0; stability tracked, direct-contract fallback retained
Multisig	@stellar-expert/tx-signers-inspector 1.8.2	
Refractor	direct REST against api.refractor.space	No npm client needed
Tests	Vitest 4.1.6, Playwright 1.60.0 with @axe-core/playwright 4.11.3, fast-check 4.8.0	Unit, browser E2E with accessibility sweep, property-based
Tooling	ESLint 9.39.4, Prettier 3.8.3, lefthook 2.1.6	Pre-commit lint, typecheck, format

6. Integration designs

6.1 Stellar Wallets Kit

Wallet connectivity runs through [@creit.tech/stellar-wallets-kit](https://github.com/creit-tech/stellar-wallets-kit) 2.3.0 behind a small [Connector](#) interface ([kind](#), [connect](#), [disconnect](#), [getPublicKey](#), [signTransaction](#), [signAuthEntry](#)), so the rest of the system signs through one abstraction. The kit implements SEP-43 and signs both classic transactions and Soroban authorization entries, which covers the classic teardown transaction and every single-call Soroban transaction. Supported modules include Freightier (which also surfaces Ledger hardware wallets through its Stellar app), xBull, Albedo, Rabet, LOBSTR, Hana, and WalletConnect v2. Connecting a wallet is the default path, which keeps raw secret-key entry, the highest-risk pattern on Stellar, off the main flow.

6.2 Blend v2 (lending)

Blend positions are read and unwound through [@blend-capital/blend-sdk](#) 3.2.2. The tool loads a user's position per pool ([PoolV2.load](#), then [loadUserPositions](#)), which returns the user's liabilities, collateral, and supply, and derives the exit sequence: if the position has liabilities, repay first, which requires holding the borrowed asset, so the tool sources it by swapping from existing balances when the user does not hold it; then withdraw collateral; then withdraw supply; then claim BLND emissions. These map to Blend's [PoolContractV2.submit](#) entry point with the corresponding [RequestType](#) values ([Repay](#), [WithdrawCollateral](#), [Withdraw](#)). Each is a single Soroban transaction submitted and confirmed before the tool re-reads state and builds the next.

Blend has no global pool registry call, so pool IDs are pulled from the published [blend-capital/blend-utils](#) deployment configuration and pinned into the contract allow-list. The mainnet allow-list covers the active pools (the Fixed and YieldBlox families across v1 and v2) and the supporting infrastructure (pool factories v1 and v2, backstops v1 and v2, the emitter, the BLND token, the USDC and XLM SACs, the Comet factory and LP, and the bootstrapper). One Blend position cannot close in a single session: a backstop deposit is subject to a 17-day withdrawal queue in Blend v2, so the tool initiates the queued withdrawal, shows the unlock date, and flags the account for a return visit rather than implying an instant exit.

6.3 Soroswap (routing, LP, and conversion)

Soroswap is integrated through [@soroswap/sdk](#) 0.4.0, which is Bearer-authenticated and proxied through the [/api/soroswap](#) server route so the API key never reaches the browser. It serves two roles. First, LP removal through [removeLiquidity](#). Second, and centrally, conversion of any non-XLM Soroban token to XLM through the Soroswap Aggregator, which routes across Soroswap, Phoenix, and Aquarius to find the best price. The Stellar DEX is

excluded from Aggregator routes because it is not Soroban-compatible, which is exactly why classic balances use native path payment instead (Section 10).

```
const quote = await soroswap.quote({
  assetIn: tokenContractId,
  assetOut: xlmSacContractId,
  amount: balance,
  tradeType: TradeType.EXACT_IN,
  protocols: [Protocol.SOROSWAP, Protocol.AQUA, Protocol.PHOENIX],
  slippageBps: '100', // 1% default; range 10-500 bps, user-adjustable
  maxHops: 2,
});
const { xdr } = await soroswap.build({ quote, from: userAddr, to: userAddr
}); the returned contract id is checked against the allow-list, then the xdr is
// re-simulated and signed client-sid
```

Mainnet Soroswap contracts pinned into the allow-list are the Router `CAG5LRYQ5JVEUI5TEID72EY0VX44TTUJT5BQR2J6J77FH65PCCFAJDDH` and the Factory `CA4HEQTL2WPEUYKYKCD0HCDNIV4QHNJ7EL4J4NQ6VADP7SYHVRYZ7AW2`. The Aggregator contracts were audited by Runtime Verification and the core AMM and Router by OtterSec. The contract address returned by any quote is verified against the allow-list before the resulting envelope is presented for signing.

6.4 Aquarius (AMM)

Aquarius has no standalone TypeScript SDK, so it is integrated by direct contract invocation through `@stellar/stellar-sdk` against its mainnet AMM router `CBQDHNBFBYE4MKPWBSJOPiYlW4SFSXAXUTSXJN76GNKYVYPCKWC6QUK`, with individual pools indexed by `pool_index: BytesN<32>`. Pool discovery uses two interchangeable providers, the public `amm-api.aqua.network` API and a direct event-scan provider, so discovery does not depend on a single source. The calls used are `withdraw(user, tokens, pool_index, share_amount, min_amounts)` to exit liquidity, `claim(user, pool_index)` for AQUA rewards, and `swap_chained(user, swaps_chain, token_in, in_amount, out_min)` where a routed swap is needed. Chained swaps are bounded at roughly four pools per chain: beyond that the call exceeds Soroban's per-transaction resource budget and fails simulation with `BudgetExceeded`, so the tool keeps routes within that bound.

6.5 FxDAO (collateralized vaults)

FxDAO is a hybrid case integrated by direct Soroban contract calls. Its synthetic stablecoins USDx, EURx, and GBPx are issued as classic Stellar assets, while the vault state lives in Soroban. Following the same liabilities-before-collateral dependency pattern as Blend, the vault-exit builder repays the vault debt and redeems collateral through Soroban calls, then

disposes of the leftover classic stablecoin on the classic side through path payment during the final classic transaction. One protocol-specific detail is that FxDAO indexes vaults in a sorted linked list, so creating or adjusting a vault requires the correct predecessor key: the adapter reads the protocol's vault index (`get_vaults_info`) and walks the sorted list to compute that key rather than guessing it. The FxDAO allow-list entries cover the vault contracts, the USDx, EURx, and GBPx SACs, the oracle, and the locking pool.

6.6 DeFi Position API: Orion / OctoPos, with direct fallback

Position discovery sits behind a single `IDeFiPositionProvider` interface with three interchangeable adapters, so the destructive logic never depends on any one external service being live:

1. **OrionProvider (primary)**: Orion (by Daccred) commits to a REST interface and names all four target protocols, normalizing positions into financial primitives. The adapter reports `provider-unavailable` cleanly until the upstream API publishes, rather than failing silently.
2. **OctoposProvider (secondary)**: OctoPos (by Untangled Finance) brings production NAV and cross-protocol pricing math, useful for Blend and Aquarius valuation.
3. **DirectContractProvider (fallback)**: Direct on-chain reads for the tier-1 protocols, aggregated with `Promise.allSettled` so one protocol timing out does not block the others: Blend via the SDK, Aquarius via per-pool share-balance queries, Soroswap via pair-contract balances, FxDAO via vault storage.

The integration is REST/JSON over HTTPS, proxied through the `/api/positions` route, which hides API keys and caches responses for a short window with a fresh fetch immediately before each exit transaction. Every response is validated against a strict Zod schema, so a malformed or divergent reply surfaces a clear mismatch rather than steering the tool with bad data. If one protocol returns and another times out, each protocol's status is shown independently and the user can proceed with the ones that resolved.

6.7 Optional: Stellar Broker

Stellar Broker is a multi-source liquidity router on the Integration List that aggregates Soroswap, Aquarius, the Stellar DEX, and classic AMMs. Because routing already sits behind a provider interface, Stellar Broker can be added as an alternative routing source for resilience and price comparison without changing the surrounding flow. Soroswap's Aggregator remains the committed primary path for Soroban-token conversion.

7. Stellar protocol primitives and operations

The tool is built on Stellar's native primitives and the official `@stellar/stellar-sdk` v14, which provides one client for both Horizon and Soroban RPC.

Reserve model: Base reserve is 0.5 XLM. An account's minimum balance is $(2 + \text{numSubEntries} + \text{numSponsoring} - \text{numSponsored}) \times \text{baseReserve} + \text{liabilities}$. A pool-share trustline (CAP-38) consumes two subentries because it implicitly references two underlying asset trustlines, which is why it must be removed before those underlying trustlines.

Classic operations used:

- `MANAGE_SELL_OFFER` / `MANAGE_BUY_OFFER` with `offerID > 0` and `amount = 0` to cancel open offers.
- `LIQUIDITY_POOL_WITHDRAW(poolID, amount, minAmountA, minAmountB)` to exit classic AMM pool shares, the minimum amounts acting as slippage guards.
- `CLAIM_CLAIMABLE_BALANCE` for balances currently claimable per their predicate; future-claimable balances raise a forfeit warning.
- `PATH_PAYMENT_STRICT_SEND` with `destMin` to convert classic non-XLM balances to XLM across the Stellar DEX and CAP-38 pools.
- `CHANGE_TRUST(limit = 0)` to remove trustlines, which requires the balance to be exactly zero, pool-share trustlines first.
- `MANAGE_DATA` with the value absent to delete data entries.
- `REVOKE_SPONSORSHIP` (CAP-33) for each sponsorship the account holds when `numSponsoring > 0`.
- `SET_OPTIONS` to remove non-master signers and reset thresholds. This is a High-threshold operation and is placed in the same final transaction as the merge.
- `CREATE_ACCOUNT` and `PAYMENT` for the mediator path to a centralized exchange.
- `ACCOUNT_MERGE`, the final High-threshold operation.

Soroban operations and authorization: Each contract call is one `InvokeHostFunctionOp`.

Authorization uses `SorobanAuthorizationEntry`:

`SOROBAN_CREDENTIALS_SOURCE_ACCOUNT` covers the common case of a classic account piggy-backing on the transaction source signature, while contract-account (smart-wallet) users are handled through `SOROBAN_CREDENTIALS_ADDRESS` with their account contract's `__check_auth`.

SEP-41 token interface: Soroban tokens implement the SEP-41 interface. The SEP-41 helper module reads `balance` and `allowance` by simulation, builds `transfer(from, to, amount)` to drain a custom token, and builds `approve(from, spender, amount, live_until_ledger)` with `amount = 0` to revoke an allowance.



```
fn balance(env: Env, id: Address) -> i128;  
fn allowance(env: Env, from: Address, spender: Address) -> i128;  
fn approve(env: Env, from: Address, spender: Address, amount: i128, live_until_ledger: u32);  
fn transfer(env: Env, from: Address, to: Address, amount: i128);
```

SAC versus custom tokens: For a classic asset used inside Soroban through its Stellar Asset Contract, a normal account's SAC balance shares the same ledger entry as the classic trustline, so removing the trustline clears it and there is nothing extra to drain. A token that exists only as a custom SEP-41 contract holds its balance in contract storage and is drained with one `transfer` call per token.

Standards: Wallet signing uses SEP-43. The canonical deployment URL is published under SEP-1 in a `stellar.toml`.

8. Account discovery and feasibility engine

Every session opens with a read-only inventory (`auditAccount(publicKey, network)`), run concurrently against Horizon and Soroban RPC.

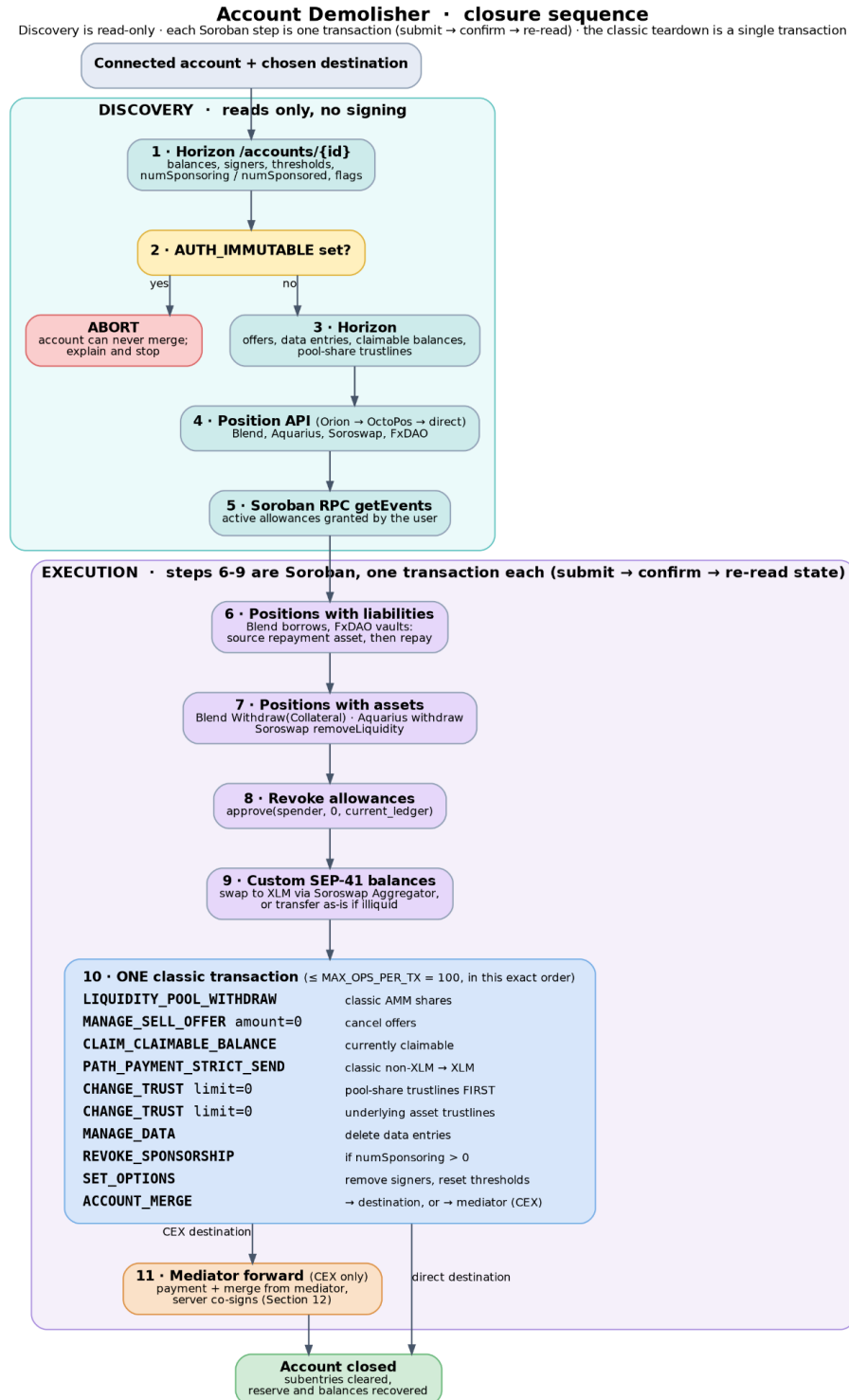
From Horizon: balances, trustlines including pool-share trustlines, open offers, data entries, signers and thresholds, sponsorship counts (`numSponsoring`, `numSponsored`), account flags including `AUTH_IMMUTABLE`, and claimable balances. Claimable balances are enumerated via `/claimable_balances?claimant={address}` and filtered by predicate, so only currently-claimable balances are auto-claimed, with a forfeit warning for any that would only become claimable in the future.

From Soroban RPC: custom token balances and DeFi positions by simulation, and active allowances by scanning `getEvents` for `approve` topics with the user's address as `from`, clamped to the RPC provider's event-retention window.

From this inventory the tool produces a feasibility verdict and stops immediately on either condition that makes a clean merge impossible: the `AUTH_IMMUTABLE` flag, and outbound sponsorship, where an account currently sponsoring reserves for others (`numSponsoring > 0`) cannot merge until those sponsorships are released. An account that is merely sponsored by another needs no special action, since the sponsor's reserve returns automatically as subentries clear. The existing tool reads neither sponsorship count and would loop indefinitely on a sponsoring account without explaining why; closing that gap is part of this work. Finally, the signer set is analyzed with `@stellar-expert/tx-signers-inspector` to determine whether the account is multisig and to select the correct signing path.

9. Execution engine and transaction model

The plan generator produces a plan tree of typed nodes in deterministic topological order. Each Soroban node is one transaction; the final classic node batches the remaining operations. The canonical order is:



When the classic operation set exceeds 100 operations, the batcher splits it across sequential transactions automatically. When a mediator is used, a small mediator-funding operation (about 2 XLM) is prepended so the mediator account exists to receive the merge.

Idempotency: Every step re-reads on-chain state before constructing the next transaction. Re-running `withdraw(amount = current_balance)` after a partial success withdraws zero; re-running `approve(amount = 0)` is a no-op; re-running `transfer(amount = current_balance)` works. The chain is the source of truth, so no client-side resumption tokens are needed. If the user closes the tab mid-flow, they reconnect, the tool re-discovers state, and the sequence resumes. Before the final classic transaction is submitted, the account is re-audited and the batch is regenerated against current state.

Orchestration: An XState v5 machine drives the flow through `discovery -> preview -> soroban-exit-loop -> multisig-collection -> final-classic-tx -> success | error-recoverable | error-fatal`, with Zustand holding global state and Zod validating user inputs.

Failure recovery is typed against real SDK error shapes, not generic retry:

Failure	Recovery
Stale sequence number	Re-fetch the account, rebuild, resubmit
Soroban resource fee underestimated	Re-simulate, raise the fee, re-sign, resubmit
Slippage exceeded	Show the new quote and require explicit user consent, not a plain retry
Footprint changed between simulation and submission	Re-simulate, re-sign, resubmit
Multisig threshold not met	Switch to partial-signature collection mode

10. Asset conversion and routing

Conversion uses a hybrid matched to the two asset types, which is the most defensible choice for a security-sensitive tool: Stellar-native path payment for the classic half (no third-party contract dependency) and a single audited aggregator for the Soroban half. Classic balances convert with `PATH_PAYMENT_STRICT_SEND`, using Horizon `/paths/strict-send` for

pathfinding and `destMin` as the slippage guard. Soroban tokens convert through the Soroswap Aggregator with an equivalent `out_min` guarantee.

Both routes are bounded by a slippage limit that defaults to 1% (100 bps), is adjustable within a 10 to 500 bps range, and is re-checked by simulation immediately before signing so price drift cannot push a swap below tolerance. For classic operations the bound is `destMin = expected × (1 - slippage)`; for Soroban swaps it is the equivalent `out_min`. This is a deliberate departure from the existing tool, which sells whatever the order book will give at a throwaway price.

A curated tier-1 asset list (XLM, USDC, EURC, AQUA, BLND, and bridged assets such as USTBL and EUTBL) guides conversion. For a token that is not tier-1 and shows more than roughly 10% aggregator price impact, or for which no path is found, the tool never sells at a loss: the user chooses to forward the balance as-is to the destination or to leave its trustline in place when converting would destroy more value than the 0.5 XLM the trustline locks.

11. Multisig and signing architecture

Multisig accounts are supported three ways, all without normalizing raw secret-key entry:

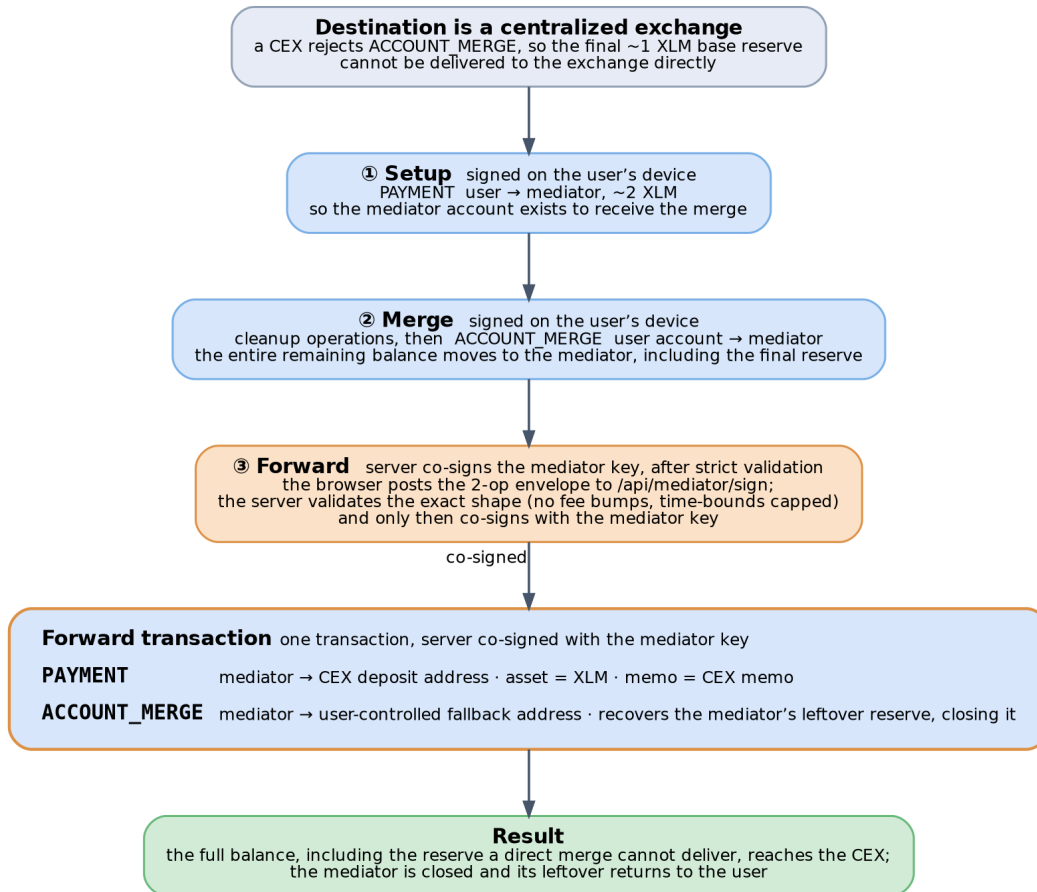
1. **Refractor:** A partially signed transaction is posted to Refractor, the StellarExpert coordination service (`api.refractor.space`), which collects the remaining signatures asynchronously so each holder signs from their own wallet in their own time. The tool subscribes for completion and submits when the threshold is met.
2. **Partial-XDR exchange:** The tool builds the canonical transaction, signs it with the first available signer, and exports the signed envelope as XDR for download or sharing. Co-signers import their signed version, and the merger combines ed25519 signatures with same-transaction-hash enforcement and an expected-signer allow-list, admitting only signatures that verify against the canonical hash. Because the network passphrase is bound into that hash, a signature for the wrong network or transaction is rejected, which also defeats cross-network replay.
3. **Sequential signing:** Several connected wallets sign in turn within one session.

Signer requirements are computed with `@stellar-expert/tx-signers-inspector`. Direct secret-key entry is retained only as an explicitly warned last resort for legacy multisig setups whose keys are spread across incompatible wallets. In that mode the seed is held in a private field in memory only, never written to `localStorage` or `IndexedDB` and never transmitted, wiped after signing, and disabled entirely by configuration. The backend never sees secret keys, never sees unsigned envelopes, and never sees wallet session tokens.

12. Mediator merge architecture (CEX destinations)

No major centralized exchange accepts an incoming `ACCOUNT_MERGE`, so a closure that targets an exchange routes through a temporary mediator account. The mediator is a per-deployment account whose seed (`MEDIATOR_SECRET`) is loaded server-side only, in a module that begins with `import "server-only"`, is memoized after first use, and is never logged. The browser fetches the mediator's public key from the signing route and bakes it into the envelopes; the user funds the mediator and merges into it; the mediator then forwards the proceeds.

Account Demolisher · mediator merge for exchange destinations
A CEX rejects `ACCOUNT_MERGE`, so the final reserve cannot reach it directly · the mediator is a temporary pass-through whose key is held server-side (per deployment) and only co-signs a strictly validated 2-op forward · the user's keys never touch the server



The validator at `lib/mediator/validator.ts` is the primary security boundary. It rejects any forward envelope that is not exactly the expected shape, with a strict rejection set covering both the merge variant and the forward variant: a fee bump, the wrong operation count, a first operation that is not `ACCOUNT_MERGE` or whose destination is not the mediator, a second operation that is not a payment or account-creation or whose source is not the mediator or whose asset is not native, missing or excessive time bounds (capped at one hour), and malformed XDR. Only after the envelope passes does the server co-sign with the mediator key.

Rate limiting (a 5-request-per-60-seconds-per-IP token bucket) and a CORS origin allow-list are defense in depth around that boundary.

The result is that the user's full balance, including the final reserve that an exchange cannot return, reaches the exchange as an ordinary memo-bearing payment, and the mediator's leftover returns to a fallback address the user controls. The user's own keys never touch the server, so the user-facing custody model is non-custodial; the only server secret is the mediator key, whose privilege is bounded to co-signing one validated envelope shape. Because the mediator key is per-deployment, a wallet or exchange runs its own mediator rather than trust a shared one. This preserves and hardens the strict-shape co-sign pattern the existing tool established.

13. Dry-run preview (the plan tree)

The RFP calls out the dry-run as the hardest UX problem, because a multi-transaction closure cannot be perfectly previewed when later transactions depend on the on-chain state produced by earlier ones. The tool's answer is an ordered, inspectable plan tree built from the audit, with one node per planned action across a set of distinct node kinds and a deterministic topological order:

1. **Static plan generation:** Enumerate the expected sequence, each node with action name, source, destination, asset, amount (or "depends on previous step"), estimated fee, and simulated output.
2. **Per-Soroban-step simulation:** For each Soroban node, `simulateTransaction` against current state to confirm it will succeed and to obtain its footprint, resource fee, and expected output. Even a transfer-as-is fallback is simulated as a real SEP-41 transfer envelope so the preview shows real fee and authorization data rather than a placeholder.
3. **Classic validation:** Assemble the final classic transaction in memory and confirm it builds, which catches malformed operations.
4. **Live re-preview:** After each Soroban transaction confirms, refresh the plan against the actual remaining state with updated outputs.

The plan renders as a timeline with per-node status icons and an expand-to-XDR drawer, so the user can audit exactly what they are about to sign. A property test asserts that the generator never produces a plan with a cycle or a step ordering that could strand the account.

14. Safety, security, and threat model

Confirmation controls: A pre-flight summary lists every operation with its estimated XLM cost and the net XLM returned. The destination is confirmed by retyping the last characters of its G-address, gated behind a short timed delay before the final merge. Accounts above a high-value threshold (for example 1000 XLM) get an additional confirmation. The memo is hard-enforced on exchange destinations: a closure to a known exchange without the required memo is refused, not merely warned.

Contract allow-list: Every Soroban transaction is checked, before signing, by `assertTransactionAllowed(tx, network)` against a hardcoded, network-specific allow-list in `lib/config/contracts.ts`. Each entry carries an id, a name, a protocol, a verification timestamp, and a source field citing the upstream snapshot it came from (for example a pinned `blend-capital/blend-utils` commit), so the list is auditable. Approximate sizes are 27 mainnet contracts (Soroswap router and factory, the Blend pools and infrastructure, the Aquarius router, and the FxDAO vaults, SACs, oracle, and locking pool) and 19 on testnet. A position-API response can never extend the list at runtime; only an edit to `contracts.ts` changes it, and a CI test greps the source to enforce that the check is present on every Soroban submission path.

Authorization scoping: Because the tool invokes contracts that themselves call `SAC transfer`, it runs `simulateTransaction` in recording-auth mode to inspect the exact `SorobanAuthorizationEntry` tree before signing, so the user's approval for one call cannot implicitly authorize unbounded downstream transfers.

Scam and fee protection: Suspected scam tokens are flagged by heuristics: lookalike symbols by edit distance, suspicious characters, off-allowlist contracts, and colliding symbols. The network fee is capped at $\max(\text{min_fee} \times 100, \text{p99} \times 2)$ read from Horizon `/fee_stats`, with a warning and a defer option during sustained surges.

Threat model: The controls map to a STRIDE-style model:

Threat	Vector	Mitigation
Destination-address substitution	Clipboard hijack, XSS	Typed last-character confirmation with timed delay; high-value warning; no <code>dangerouslySetInnerHTML</code>
Phishing clone domain	DNS, search ads, lookalikes	Canonical URL in <code>stellar.toml</code> under SEP-1; build-provenance attestation on every release

Threat	Vector	Mitigation
Supply-chain compromise	npm package takeover	Committed lockfile installed frozen in CI; a CI grep test that no analytics SDK is present; provenance attestation
Malicious position-API response	Backend takeover	Strict Zod response schema; hardcoded contract allow-list verified at sign time
Mediator-key leakage	XSS, accidental logging	Server-only memoized secret, never logged; validator rejects fee bumps and caps time bounds at one hour
Swap front-running / MEV	Public execution	Slippage bound re-checked at sign time; slippage-exceeded requires explicit user consent
Partial-XDR replay	Cross-network	Network passphrase bound into the transaction hash; merger admits only signatures verifying against it
Reentrancy on exit	Hostile protocol	Soroban host disallows reentrancy; auth tree inspected pre-sign

Content-security policy: The application runs under a strict CSP. The most security-relevant directive is `connect-src`, pinned to exactly the upstream endpoints the code talks to (three Horizon, three Soroban RPC, Refractor, two Aquarius, Soroswap, and two Friendbot for testnet helpers); anything else is a violation. `object-src` is `none`, `frame-ancestors` is `none`, `base-uri` and `form-action` are `self`, and there is no `dangerouslySetInnerHTML` anywhere. `script-src` is `'self'` plus `'wasm-unsafe-eval'` for the SDK's ed25519 WASM module, plus `'unsafe-inline'`, which Next.js 16 App Router requires because it streams the React Server Components payload as inline bootstrap scripts; a per-request nonce is incompatible with the year-long static caching of the pre-rendered pages. `script-src` still blocks every third-party script origin and remote `eval`, so what it permits is only the framework's own inline boot payload.

Deployment hardening: Dependencies are version-pinned and CI-frozen; releases carry a build-provenance attestation through an SLSA-style workflow; static pages are served from a CDN with an optional IPFS mirror; and the canonical URL is published in `stellar.toml` under SEP-1.

Audit: A security audit runs through Audit Bank, whose credits unlock at Testnet completion. The audit scope, prepared as an audit-scope document with the STRIDE matrix and in-and-out-of-scope modules, covers mediator-key generation and lifecycle, the merge-envelope validator, plan-generation correctness (no sequence that could strand an account), transaction and signature handling, and an SDK supply-chain review. The audit itself is not part of the requested budget; only the engineering time to remediate findings is.

15. Backend service

The backend is intentionally thin: three Next.js API routes on the Node runtime, each gated by an in-process 5-request-per-60-seconds-per-IP token bucket. `/api/mediator/sign` validates the two-operation envelope (merge or forward variant) and co-signs with the mediator key; `/api/positions` and `/api/soroswap` are Bearer-auth-concealing proxies so the upstream API keys never reach the browser. It holds no database, no user sessions, and no analytics. The only required server secret is `MEDIATOR_SECRET`; the positions and Soroswap API keys are optional, and their absence is reported as `provider-unavailable` rather than failing silently.

Read endpoints used:

Source	Endpoints	Used for
Horizon	<code>/accounts/{id}</code> , <code>/accounts/{id}/offers</code> , <code>/accounts/{id}/data</code> , <code>/claimable_balances?claimant=...</code> , <code>/paths/strict-send</code> , <code>/fee_stats</code>	Classic state, claimable balances, classic pathfinding, fee-surge detection
Soroban RPC	<code>simulateTransaction</code> , <code>getEvents</code> , <code>getLedgerEntries</code> , <code>sendTransaction</code> , <code>getTransaction</code>	Contract reads, allowance event scans, simulation, submission, confirmation

Endpoint configuration is exposed in the UI so users can point at self-hosted Horizon and RPC providers, and the `connect-src` allow-list is the enforcement boundary for which endpoints the client may reach.

16. Testing strategy

Unit tests (Vitest): Plan generation, position-to-operation mapping, mediator orchestration, routing and slippage math, XDR construction and parsing, allowance enumeration, and the per-protocol adapters.

Integration tests on testnet: Disposable accounts are created via friendbot and pre-populated with target profiles, then demolished end-to-end with a post-state assertion that the account is deleted. Profiles include a high-subentry classic account (many trustlines, offers, and data entries), an active Blend lending-and-borrowing position, Aquarius LP shares across multiple pools, Soroswap LP shares, sponsored entries in both the sponsored-by and sponsoring cases, a 2-of-3 multisig with varied thresholds, and a Soroban-heavy account with minimal classic trustlines.

Adversarial and edge cases: `AUTH_IMMUTABLE` refusal, sponsoring refusal, deauthorized issuers, hostile or non-compliant SEP-41 implementations, accounts with more than 50 and more than 100 subentries (to exercise discovery and the 100-operation split), deeply nested claimable-balance predicates, and race conditions where another party mutates account state mid-flow.

Soroban contract integration: Mock contracts deployed with `soroban-sdk` test utilities, and a full Blend stack via the Blend contract SDK's fixture for end-to-end Blend exit tests.

Browser end-to-end (Playwright with axe-core): A fake-wallet module exercises the wallet-connect happy path, a full demolition, each error-recovery path, multisig XDR import and export, and the secret-key fallback, with an accessibility sweep that must report zero violations on every route.

Property-based fuzzing (fast-check): The plan generator is run against many randomized account states, asserting a fixed set of properties: every plan is valid XDR, operations are in correct threshold order, dependencies are respected, and no sequence could strand an account. Cycle detection accompanies it.

17. Deployment and verification

The trust-critical pages are statically pre-rendered, which keeps hosting cheap and allows a censorship-resistant IPFS mirror. The build is deployed under a canonical domain controlled by the project, with the URL published in that domain's `stellar.toml` under SEP-1 and served from a CDN, so users can verify they are on the genuine tool before trusting it with an irreversible action. Releases are signed with a build-provenance attestation, and the dependency lockset is pinned and CI-verified. A reference instance is operated for users, and the open-source build plus the per-deployment mediator let wallets and exchanges self-host their own instance.

18. Forward-looking robustness

The architecture is built to fail safely as the protocol and the ecosystem move. If the tool encounters a subentry type or account state it does not recognize, it refuses to proceed and surfaces an explicit error rather than attempting a merge in an unknown state. The contract allow-list is anchored to pinned upstream snapshots, and a CI job that diffs each protocol's published deployment list flags redeployments before they reach users. Allowance enumeration depends on the Soroban event stream, so the design tracks the Protocol 26 (Yardstick) changes to the eviction and event-scan model and is exercised against both the current and the updated event shape. Two testnet-only constraints are handled as explicit diagnostics rather than silent failures: Soroswap currently has no on-chain testnet liquidity, so testnet swap quotes return no path and surface a clear message while the mainnet path is unaffected, and the FxDAO testnet stablecoin issuer is intermittently absent upstream, so the testnet vault round-trip is gated on its availability while the mainnet path is unaffected.

19. Integration effort and milestone mapping

The build runs four months, one developer, on the standard SCF 10/20/30/40 schedule. Integration work is concentrated where the Integration List signals the most effort.

Milestone	Integration work	Relevant Integration List building blocks
MVP (T1)	Wallet connection and signing; full classic discovery, teardown, and the mediator-signing service	Stellar Wallets Kit (listed at under 1 day to connect, extended here into the full signing and multisig flow)
Testnet (T2)	The DeFi differentiator: Blend, Aquarius, Soroswap, and FxDAO unwinding; the position-discovery and routing layer; the dry-run plan tree, multisig coordination, and safety suite	Blend v2 (1+ month), Soroswap (1-2 weeks), Aquarius (under 1 day), plus the Orion / OctoPos position API
Mainnet (T3)	Production hardening, the Audit Bank audit and remediation, the full test suite, documentation, and launch	Finalized contract allow-list across all integrated protocols

Blend v2 carries the largest single integration effort, consistent with the Integration List's 1+ month estimate, because of the repay-before-withdraw dependency, the emissions claim, and the 17-day backstop queue. The Soroswap and Aquarius integrations are the routing and AMM-exit backbone, and the position API plus direct fallback ties discovery together. The MVP front-loads the classic path and the Stellar Wallets Kit integration so the tool is usable end-to-end before the DeFi surface lands.

20. References

SCF Build Integration List:

<https://stellar.gitbook.io/scf-handbook/scf-awards/scf-build/integration-track/integration-list>

Integrated building blocks (docs):

- Stellar Wallets Kit: [@creit.tech/stellar-wallets-kit](https://github.com/creit/stellar-wallets-kit)
- Blend v2: [@blend-capital/blend-sdk](https://github.com/blend-capital/blend-sdk); pool configs in [blend-capital/blend-utils](https://github.com/blend-capital/blend-utils)
- Aquarius: docs.aqua.network; [amm-api.aqua.network](https://github.com/amm-api); AMM router [CBQDHNBFBYE4MKPWBSJOPIYLW4SFSXAXUTSXJN76GNKYVYPCKWC6QUK](https://github.com/CBQDHNBFBYE4MKPWBSJOPIYLW4SFSXAXUTSXJN76GNKYVYPCKWC6QUK)
- Soroswap: [@soroswap/sdk](https://github.com/soroswap/sdk); docs.soroswap.finance; Router [CAG5LRYQ5JVEUI5TEID72EY0VX44TTUJT5BQR2J6J77FH65PCCFAJDDH](https://github.com/CAG5LRYQ5JVEUI5TEID72EY0VX44TTUJT5BQR2J6J77FH65PCCFAJDDH), Factory [CA4HEQTL2WPEUYKYKCD0HCDNIV4QHNJ7EL4J4NQ6VADP7SYHVRYZ7AW2](https://github.com/CA4HEQTL2WPEUYKYKCD0HCDNIV4QHNJ7EL4J4NQ6VADP7SYHVRYZ7AW2)
- FxDAO: app.fxdao.io
- DeFi Position APIs: Orion (orion.daccred.co), OctoPos (stellar.untangled.finance)
- Refractor: api.refractor.space

Protocol standards and primitives: SEP-1 (stellar.toml), SEP-41 (token interface), SEP-43 (wallet signing), CAP-33 (sponsored reserves), CAP-38 (liquidity pools and pool-share trustlines), and the [ACCOUNT_MERGE](https://developers.stellar.org), [SET_OPTIONS](https://developers.stellar.org), [PATH_PAYMENT_STRICT_SEND](https://developers.stellar.org), [LIQUIDITY_POOL_WITHDRAW](https://developers.stellar.org), [CHANGE_TRUST](https://developers.stellar.org), [MANAGE_DATA](https://developers.stellar.org), [MANAGE_SELL_OFFER](https://developers.stellar.org), [CLAIM_CLAIMABLE_BALANCE](https://developers.stellar.org), and [REVOKE_SPONSORSHIP](https://developers.stellar.org) operations documented at developers.stellar.org.